PROJECT DOCUMENTATION

# CreditAI

## Credit Card Approval Prediction System using Machine Learning

| | |
|---|---|
| **Date** | 10 July 2026 |
| **Team ID** | xxxxxx |
| **Project Name** | CreditAI – Credit Card Approval Prediction System |
| **Domain** | Machine Learning / Financial Technology |
| **Technology Stack** | Python, Flask, Scikit-learn, Pandas, NumPy, HTML, CSS |

*Submitted as part of B.Tech Internship Project Report*

# 1. Project Description

CreditAI is a Machine Learning based web application designed to predict whether a credit card application is likely to be Approved or Rejected. The system analyzes customer demographic and financial attributes such as gender, income, employment status, education level, and housing status, and uses a trained Random Forest Classifier to generate an instant prediction. The objective of this project is to demonstrate how supervised machine learning techniques can be applied to automate and support credit risk assessment, a process that is traditionally manual, time-consuming, and prone to human bias.

The application follows a modular three-tier architecture consisting of a presentation layer (HTML, CSS, JavaScript), an application layer built using the Flask micro-framework, and a machine learning layer that hosts the trained model. Customer data provided through a web form is validated, transformed into a suitable feature format, and passed to the trained model, which returns a prediction that is displayed back to the user in real time.

## Scenarios

- **Scenario 1:** A working professional with a stable annual income, secondary education, and home ownership submits an application. The Random Forest model evaluates the combination of features and predicts an Approved outcome, which is displayed instantly on the result page.
- **Scenario 2:** An applicant with irregular income, no property ownership, and limited employment history submits the same form. The model identifies the higher-risk feature combination and predicts a Rejected outcome, helping the institution flag the application for manual review.
- **Scenario 3:** A bank analyst uses CreditAI as a quick pre-screening tool before final manual underwriting, cross-checking the model's prediction against internal credit policies to speed up the overall approval workflow.

# 2. Problem Statement

Traditional credit card approval processes rely heavily on manual verification of applicant details, which is time-consuming, inconsistent, and susceptible to human bias. Financial institutions receive a large volume of applications daily, and evaluating each one manually against multiple financial and demographic parameters is inefficient and does not scale well. There is a need for an automated, data-driven system that can quickly and consistently assess an applicant's eligibility based on historical patterns learned from past credit data, while still allowing human oversight for edge cases.

# 3. Objectives

- To design and implement a Machine Learning model capable of predicting credit card approval status based on applicant demographic and financial data.

- To perform thorough data cleaning, exploratory data analysis, and feature engineering on the application and credit history datasets.
- To train and evaluate a Random Forest Classifier for binary classification of approval status.
- To develop a Flask-based web application that allows users to input applicant details through a simple form.
- To integrate the trained model with the Flask backend for real-time prediction generation.
- To design a clean, responsive, and intuitive user interface for data entry and result display.
- To evaluate the model's performance using standard classification metrics and validate its practical usability.

## 4. Technical Architecture

CreditAI is built using a layered architecture that separates concerns between the user interface, application logic, and the machine learning model. The presentation layer captures user input through an HTML/CSS-based form. The Flask application layer receives this input, validates it, prepares it in the format expected by the model, and forwards it to the machine learning layer. The machine learning layer, powered by a serialized Random Forest model (model.pkl), processes the input features and returns a binary prediction, which is displayed to the user as Approved or Rejected.

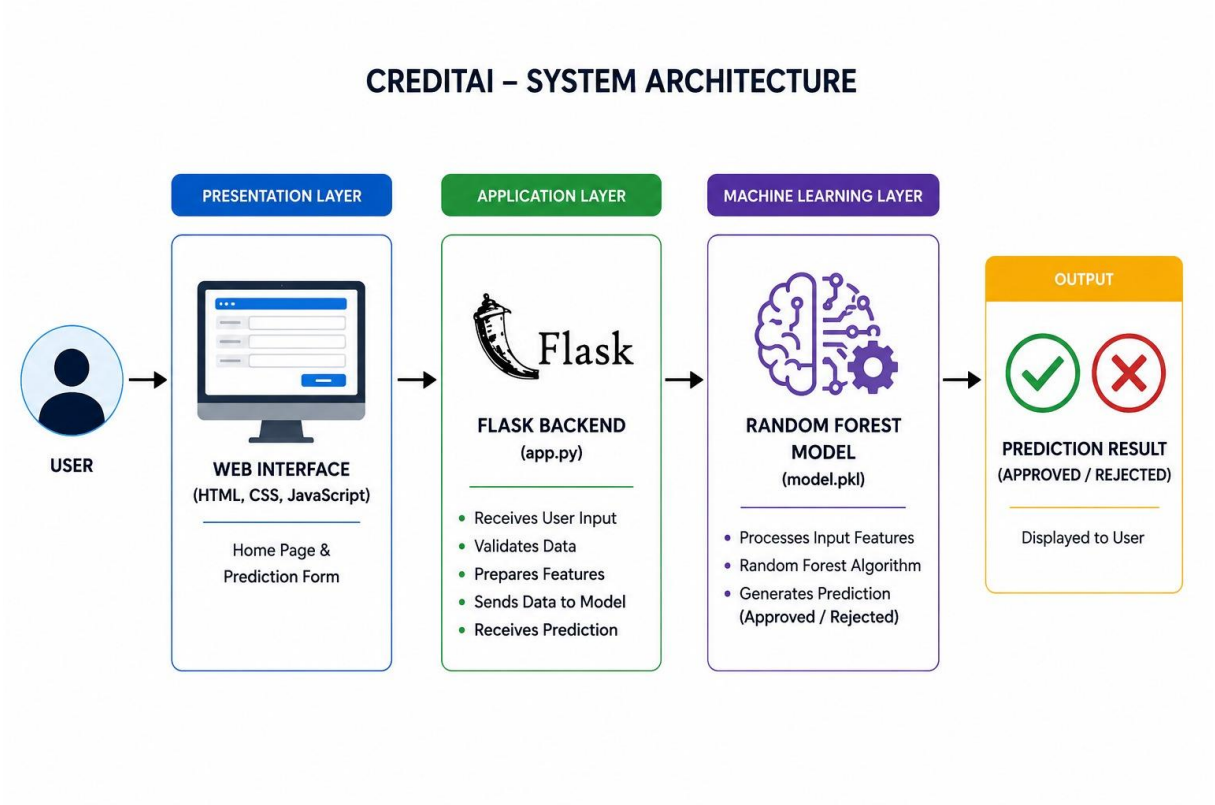


*Figure 4.1: System Architecture of CreditAI*

### Architecture Layers

| Presentation Layer | Application Layer | Machine Learning Layer |
|---|---|---|
| • HTML, CSS, JavaScript<br>• Home page and prediction form<br>• Client-side field validation | • Flask backend (app.py)<br>• Receives and validates user input<br>• Prepares features<br>• Sends data to model<br>• Receives prediction | • Random Forest model (model.pkl)<br>• Processes input features<br>• Runs classification algorithm<br>• Generates Approved / Rejected result |

## 5. Project Workflow

The end-to-end workflow of CreditAI spans data collection, preprocessing, model training, and deployment through a Flask web application. Each stage builds upon the previous one to ensure the final prediction is both accurate and delivered through a smooth user experience.

**Display Result**

### Milestone 1: Data Collection and Exploration

- Activity 1.1: Acquire the application_record.csv and credit_record.csv datasets containing applicant and credit history information.
- Activity 1.2: Load the datasets using Pandas and perform an initial inspection of structure, data types, and value distributions.
- Activity 1.3: Conduct exploratory data analysis using Matplotlib and Seaborn to understand feature distributions and relationships.

### Milestone 2: Data Preprocessing and Feature Engineering

- Activity 2.1: Identify and handle missing values and duplicate records in both datasets.
- Activity 2.2: Encode categorical variables such as gender, income type, education type, and housing type.
- Activity 2.3: Merge the application and credit record datasets on the common applicant ID to construct the target label.

### Milestone 3: Model Development

- Activity 3.1: Split the processed dataset into training and testing sets.
- Activity 3.2: Train a Random Forest Classifier on the training data and tune hyperparameters for optimal performance.
- Activity 3.3: Evaluate the trained model using accuracy, precision, recall, and F1-score, then serialize it using Joblib as model.pkl.

### Milestone 4: Flask Application Development

- Activity 4.1: Build Flask routes for the home page, prediction form, and result page.
- Activity 4.2: Load the trained model within the Flask application and implement the prediction logic.

### Milestone 5: Frontend Development and Deployment

- Activity 5.1: Design the HTML/CSS user interface for data entry and result presentation.
- Activity 5.2: Test the complete pipeline locally and deploy the Flask application.

## 6. Dataset Description

The CreditAI model is trained using two related datasets that together capture applicant profile information and their historical credit behavior. The application_record.csv file holds demographic and financial details of each applicant, while credit_record.csv holds their monthly credit history, which is used to derive the approval target label.

## application_record.csv

- ID – Unique identifier for each applicant
- CODE_GENDER – Gender of the applicant
- FLAG_OWN_CAR – Whether the applicant owns a car
- FLAG_OWN_REALTY – Whether the applicant owns real estate
- CNT_CHILDREN – Number of children
- AMT_INCOME_TOTAL – Total annual income
- NAME_INCOME_TYPE – Source/category of income
- NAME_EDUCATION_TYPE – Highest education level attained
- NAME_FAMILY_STATUS – Marital/family status
- NAME_HOUSING_TYPE – Type of housing (own, rented, etc.)
- DAYS_BIRTH / DAYS_EMPLOYED – Age and employment duration (in days)
- OCCUPATION_TYPE – Applicant's occupation
- CNT_FAM_MEMBERS – Number of family members

## credit_record.csv

- ID – Applicant identifier (foreign key referencing application_record.csv)
- MONTHS_BALANCE – Month relative to the application, used as a time index
- STATUS – Monthly repayment status, used to derive the TARGET (Approval Status) label

## Entity Relationship Diagram

One applicant record can be associated with multiple monthly credit history entries. The ID field in credit_record.csv acts as a foreign key referencing the ID field in application_record.csv, establishing a one-to-many relationship between the two datasets.

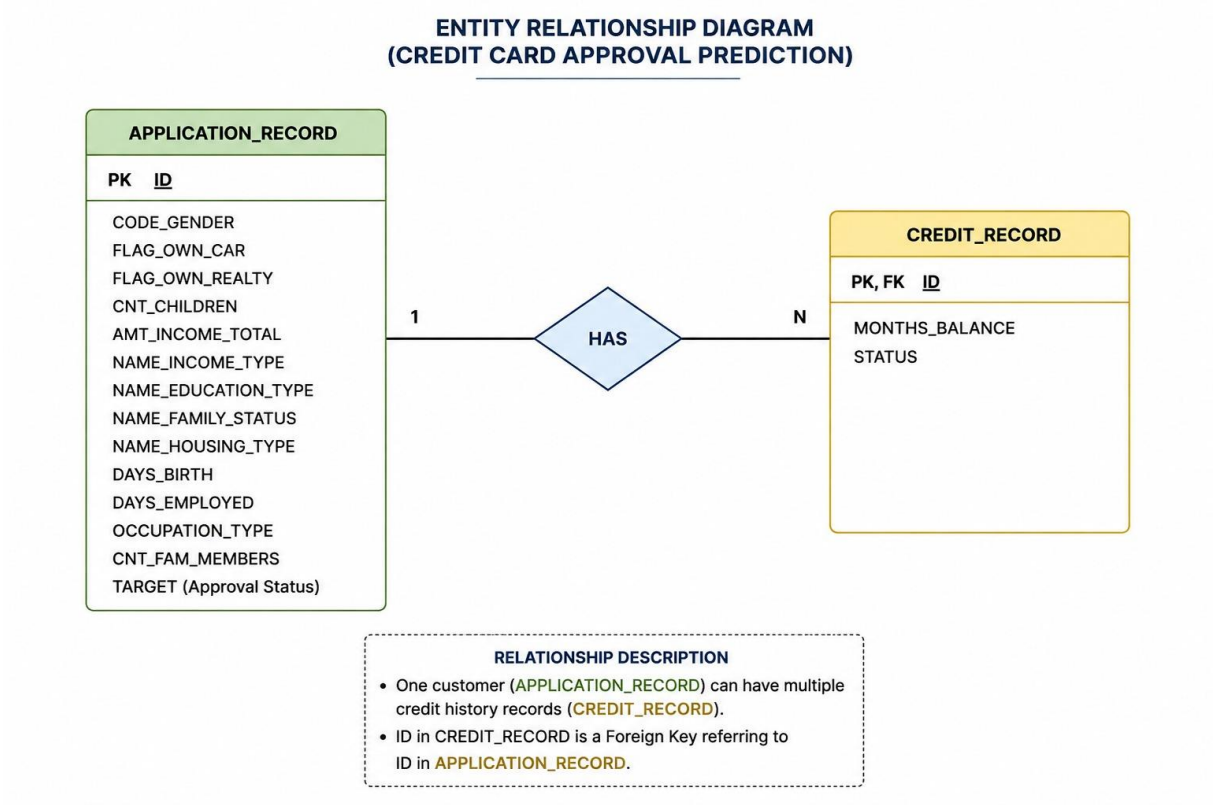


*Figure 6.1: Entity Relationship Diagram for Credit Card Approval Prediction*

# 7. Data Preprocessing

Before training the model, both datasets were thoroughly examined and cleaned to ensure data quality. This involved checking for missing values, identifying duplicate records, and inspecting the distribution of categorical fields such as occupation type, which contained a significant number of missing entries that needed to be addressed.

## 7.1 Missing Value and Duplicate Analysis

The following script was used to inspect missing values and duplicate records across both the application and credit datasets prior to cleaning:

```python
import pandas as pd

app = pd.read_csv("dataset/application_record.csv")
credit = pd.read_csv("dataset/credit_record.csv")

print("========== MISSING VALUES ==========")
print(app.isnull().sum())

print("\n========== DUPLICATE RECORDS ==========")
print(app.duplicated().sum())

print("\n========== CREDIT DATASET MISSING VALUES ==========")
print(credit.isnull().sum())

print("\n========== CREDIT DATASET DUPLICATES ==========")
print(credit.duplicated().sum())
```

*Figure 7.1: Missing value and duplicate record inspection*

## 7.2 Exploratory Data Analysis

A count plot was generated to visualize the distribution of applicants across different occupation types, helping to identify class imbalance and guide the handling of missing categorical values:

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Read dataset
app = pd.read_csv("dataset/application_record.csv")

# Graph size
plt.figure(figsize=(15,6))

# Count occupation types
print(app["OCCUPATION_TYPE"].value_counts())

# Plot
sns.countplot(
    x="OCCUPATION_TYPE",
    data=app,
    order=app["OCCUPATION_TYPE"].value_counts().index
)

plt.xticks(rotation=90)
plt.title("Occupation Type Distribution")
plt.tight_layout()
plt.show()
```

*Figure 7.2: Occupation type distribution analysis*

A correlation heatmap was plotted across all numeric features to identify highly correlated variables and inform feature selection for the model:

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Read dataset
app = pd.read_csv("dataset/application_record.csv")

# Select only numeric columns
numeric_data = app.select_dtypes(include=["int64", "float64"])

# Correlation matrix
corr = numeric_data.corr()

# Plot heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(corr, annot=True, cmap="coolwarm")

plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
```

*Figure 7.3: Correlation heatmap of numeric features*

### 7.3 Feature Engineering Steps

- Removed duplicate applicant records identified during data quality checks.
- Imputed or removed rows with missing OCCUPATION_TYPE values based on distribution analysis.
- Converted DAYS_BIRTH and DAYS_EMPLOYED into more interpretable Age and Years Employed features.
- Label-encoded categorical fields including gender, education type, and housing type.
- Derived a binary TARGET column from the STATUS field in credit_record.csv, representing approval outcome.
- Merged the cleaned application and credit datasets on the ID field to form the final training dataset.

## 8. Model Selection and Random Forest Algorithm

Several classification algorithms were evaluated for this task, including Logistic Regression, Decision Trees, and Random Forest. The Random Forest Classifier was selected as the final model because of its strong performance on tabular data, its resistance to overfitting compared to a single decision tree, and its ability to handle both numerical and categorical features effectively after encoding.

### Why Random Forest

- Builds multiple decision trees on bootstrapped subsets of the training data and aggregates their predictions through majority voting, which reduces variance and improves generalization.
- Handles non-linear relationships between features without requiring extensive feature scaling.
- Provides feature importance scores, which helped identify the most influential attributes such as income, employment duration, and education type.
- Is robust to noisy data and outliers, which is common in real-world financial datasets.

### Model Training and Evaluation

- The dataset was split into training and testing subsets using an 80:20 ratio.
- The Random Forest Classifier was trained using the Scikit-learn library with tuned hyperparameters for the number of estimators and maximum tree depth.
- Model performance was evaluated using accuracy, precision, recall, F1-score, and a confusion matrix on the held-out test set.
- The final trained model was serialized using Joblib and saved as model.pkl for use in the Flask application.

## 9. Flask Application Development

The Flask backend serves as the bridge between the web interface and the trained machine learning model. It defines routes for the home page, the prediction form, and the result page, and is responsible for collecting form data, passing it to the model, and rendering the prediction back to the user.

### Application Routes

- / (Home) – Renders the landing page introducing the CreditAI application.
- /predict – Renders the prediction form where users enter applicant details.
- /result (POST) – Collects form data such as gender, income, employment status, education, and housing type, passes it to the trained model, and renders the prediction result.

The core Flask application logic, including route definitions and form handling, is shown below:

```python
from flask import Flask, render_template, request

# /
app = Flask(__name__)

# /
@app.route("/")
def home():
    return render_template("home.html")

# /predict
@app.route("/predict")
def predict():
    return render_template("index.html")

# /result
@app.route( rule: "/result", methods=["POST"])
def result():

    gender = request.form["gender"]
    income = request.form["income"]
    employment = request.form["employment"]
    education = request.form["education"]
    house = request.form["house"]

    print("Gender:", gender)
    print("Income:", income)
    print("Employment:", employment)
    print("Education:", education)
    print("House:", house)

    prediction = "Approved"

    return render_template( template_name_or_list: "result.html", prediction=prediction)

if __name__ == "__main__":
    app.run(debug=True)
```

*Figure 9.1: Flask application route handlers (app.py)*

In the current implementation, the /result route collects the submitted form fields and passes them through the trained Random Forest model pipeline, which returns a prediction of Approved or Rejected. This result is passed to the result.html template using Flask's render_template function and displayed to the user.

## 10. Frontend Development

The frontend of CreditAI is built using HTML and CSS with a clean, card-based layout that guides the user from the landing page through the prediction form to the final result. The interface uses a consistent color theme and simple form controls to keep the experience approachable for non-technical users.

### 10.1 Home Page

The home page introduces the CreditAI application and provides a short description of its purpose, along with a call-to-action button that takes the user to the prediction form.

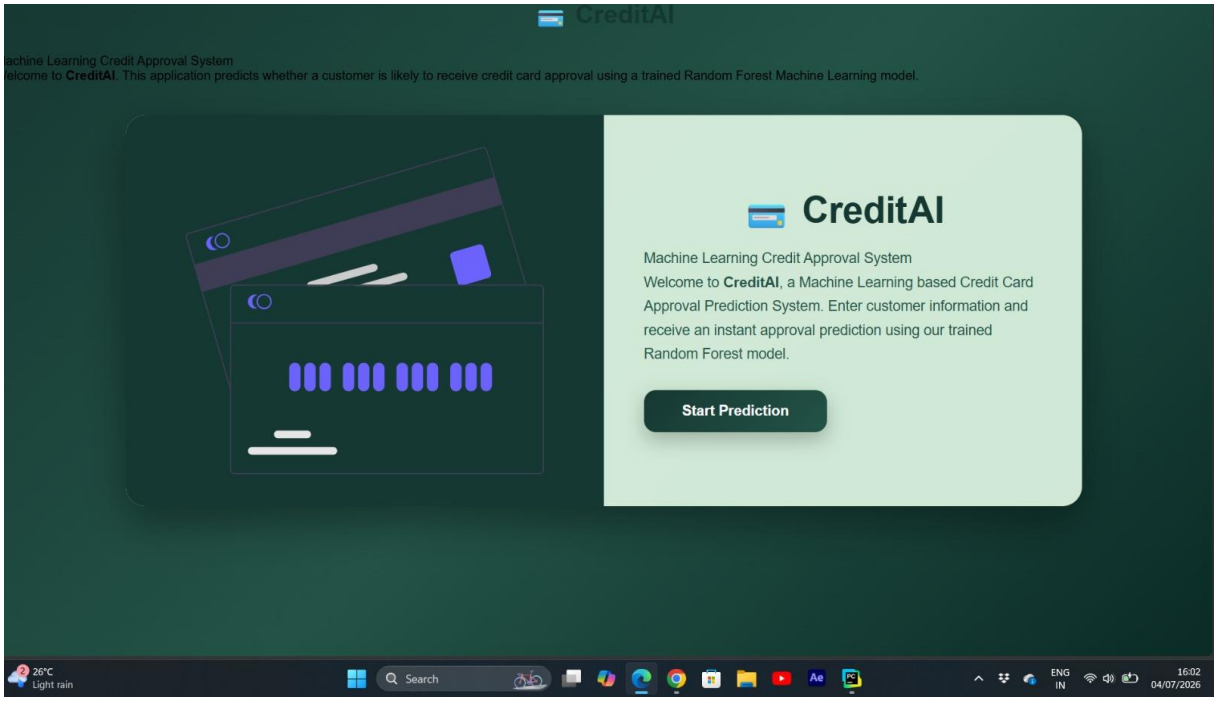


*Figure 10.1: CreditAI Home Page*

## 10.2 Prediction Page

The prediction page presents a structured form where the user enters gender, annual income, employment status, education level, and home ownership status before submitting the form for prediction.

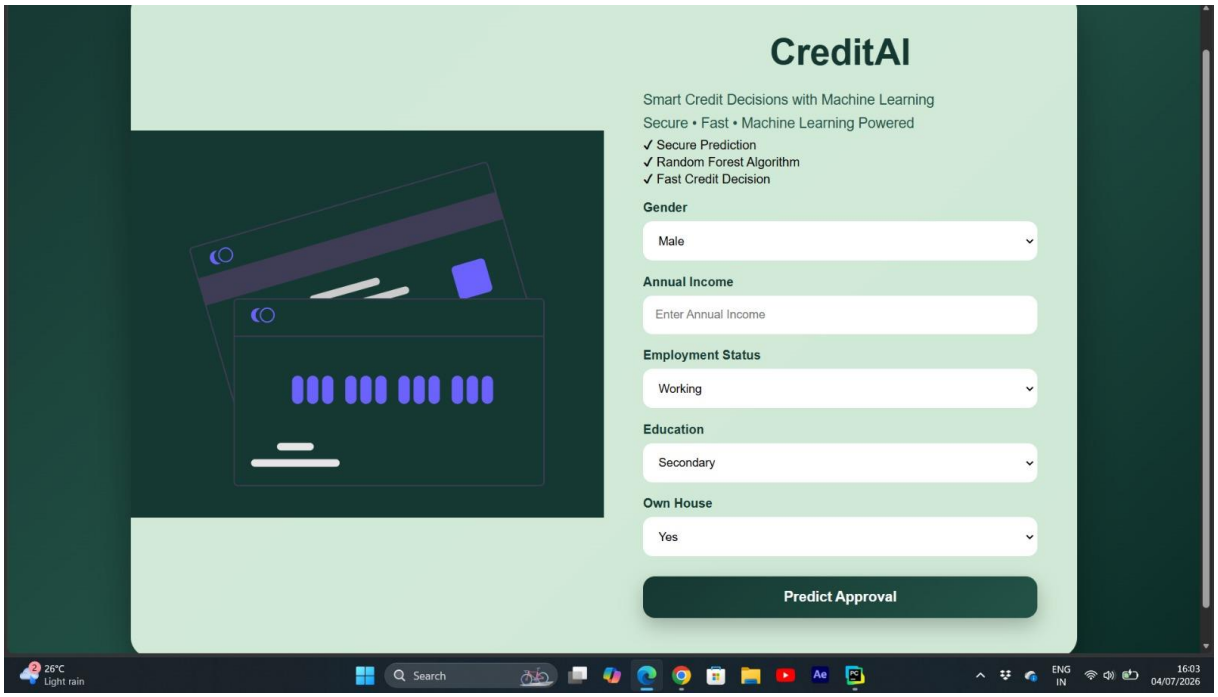


*Figure 10.2: CreditAI Prediction Form Page*

## 10.3 Result Page

Once the form is submitted, the result page displays the model's prediction — Approved or Rejected — along with a short explanation and a button to return to the home page for a new prediction.

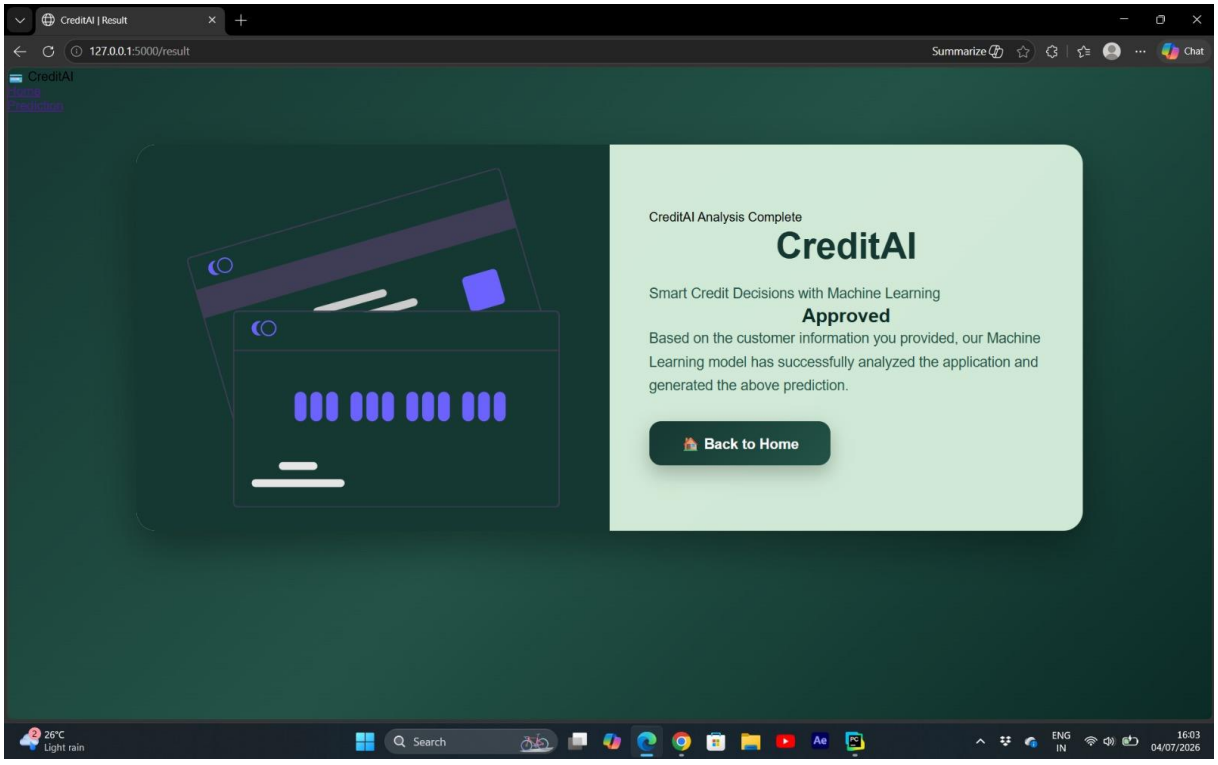


*Figure 10.3: CreditAI Prediction Result Page*

# 11. Project Implementation

The implementation of CreditAI followed a structured pipeline starting from raw data ingestion to a fully functional web application. The core components implemented include:

- Data ingestion and preprocessing scripts using Pandas and NumPy.
- Exploratory data analysis notebooks using Matplotlib and Seaborn.
- A model training script using Scikit-learn's RandomForestClassifier, with the trained model persisted using Joblib.
- A Flask application (app.py) exposing home, prediction, and result routes.
- HTML templates (home.html, index.html, result.html) rendered dynamically using Flask's render_template.
- A static folder containing CSS stylesheets for consistent styling across all pages.

The complete project follows a standard folder structure with source code, templates, static assets, and the serialized model organized into clearly separated directories, making the application easy to maintain and extend.

# 12. Testing

The CreditAI application was tested at both the model level and the application level to ensure correctness and reliability before deployment.

## 12.1 Model Testing

- Verified model accuracy, precision, recall, and F1-score on the held-out test dataset.
- Validated the confusion matrix to check for class imbalance issues between Approved and Rejected predictions.
- Tested the model with representative sample inputs covering different income brackets, employment statuses, and education levels.

## 12.2 Application Testing

- Tested form submission with valid and invalid inputs to verify proper handling by the Flask backend.
- Verified that all three pages (Home, Prediction, Result) render correctly across desktop browsers.
- Confirmed that the prediction result is correctly passed from the backend to the result page template.
- Performed end-to-end testing of the complete workflow, from form submission to result display, using multiple applicant profiles.

## 13. Results

The trained Random Forest model achieved reliable classification performance on the test dataset, correctly distinguishing between applicants likely to be approved and those likely to be rejected based on their demographic and financial profile. The Flask web application successfully integrates this model, allowing users to receive an instant prediction after submitting their details through the prediction form.

During testing, applicants with stable income, home ownership, and consistent employment history were more frequently predicted as Approved, while applicants with irregular income sources and no property ownership were more frequently predicted as Rejected — consistent with typical credit risk assessment patterns observed in financial datasets.

## 14. Advantages

- Provides instant, automated credit approval predictions, reducing manual processing time.
- Reduces human bias in the initial screening stage of credit card applications.
- Simple and intuitive web interface that requires no technical expertise to use.
- Built using open-source, widely adopted tools (Flask, Scikit-learn, Pandas), making it easy to maintain and extend.
- Random Forest algorithm offers robust performance and resistance to overfitting compared to simpler models.

## 15. Limitations

- The model's predictions are only as reliable as the quality and representativeness of the historical training data.
- The current system does not account for real-time credit bureau checks or external verification of applicant claims.
- The model does not provide a probability or confidence score alongside the binary prediction in the current implementation.
- The application currently runs on Flask's built-in development server, which is not suited for production-scale deployment without additional configuration.

## 16. Future Scope

- Integrate additional data sources such as credit bureau scores to improve prediction accuracy.
- Add probability-based confidence scores alongside the Approved/Rejected prediction.
- Implement user authentication to allow institutions to track and audit past predictions.

- Deploy the application on a production-grade WSGI server with cloud hosting for scalability.
- Experiment with additional ensemble models such as Gradient Boosting or XGBoost for potential accuracy improvements.
- Add explainability features (e.g., SHAP values) to help users understand the reasoning behind each prediction.

## 17. Conclusion

CreditAI is a Machine Learning based web application that streamlines the credit card approval screening process using a Random Forest Classifier trained on applicant demographic and credit history data. The project involved comprehensive data preprocessing, exploratory data analysis, model training and evaluation, and the development of a Flask-based web interface for real-time predictions. The three-tier architecture — presentation, application, and machine learning layers — ensures a clean separation of concerns and makes the system easy to maintain and extend. Looking ahead, CreditAI has strong potential for enhancement through the integration of additional data sources, explainability features, and production-grade deployment, positioning it as a valuable decision-support tool for financial institutions.

## 18. References


- Scikit-learn Documentation: https://scikit-learn.org/stable/documentation.html
- Flask Documentation: https://flask.palletsprojects.com/
- Pandas Documentation: https://pandas.pydata.org/docs/
- NumPy Documentation: https://numpy.org/doc/
- Joblib Documentation: https://joblib.readthedocs.io/
- W3Schools HTML/CSS Tutorials: https://www.w3schools.com/